9)
Explain the difference between `git pull`, `git fetch`, and `git clone`. When would you use each?

Ans:

Command	explanation	When to use
Git clone	Creates a complete copy of a remote Git repository on your local computer, including all files, branches, and commit history.	Use it when you are downloading a repository for the first time.
Git fetch	Downloads the latest changes from the remote repository but does not merge them into your current branch. It lets you review updates before applying them.	Use it when you want to check for new changes without affecting your current work.
Git pull	Downloads the latest changes from the remote repository and automatically merges them into your current branch.	Use it when you want to update your local repository with the latest changes from the remote repository

Summary:

- **`git clone`** → First-time download of a repository.
- **`git fetch`** → Check and download updates without merging.
- **`git pull`** → Download and immediately merge updates into your current branch.

10) What is a .gitignore file? Write a .gitignore for an Arduino project that excludes compiled output files (.hex, .elf), OS-specific files (.DS_Store, Thumbs.db), and IDE config folders (.vscode/, build/).

Ans :

A **`.gitignore`** file tells Git which files and folders should not be tracked or uploaded to the repository. It is used to exclude generated files, temporary files, operating system files, and IDE-specific configuration files.

Example `.gitignore` for an Arduino project:

```
# Compiled output
*.hex
*.elf

# Build folder
build/

# VS Code
.vscode/

# macOS
.DS_Store

# Windows
Thumbs.db
```

11) IoT Architecture

Draw and label a 4-layer IoT architecture diagram: Perception Layer, Network Layer, Processing Layer, and Application Layer. Give one example device/technology for each layer

Ans :

layer	function	examples
Perception Layer	Collects data from the environment using sensors and actuators.	DHT11 Temperature Sensor
Network Layer	Transfers data between devices and the cloud.	Wi-Fi, Bluetooth, MQTT
Processing layer	Processes, stores, and analyzes the received data.	Cloud Server, Database
Application layer	Provides services and user interaction.	Smart Home Mobile App

Q12. Microcontroller vs Microprocessor

Ans:

parameter	Arduino uno	Raspberry pi
CPU Speed	16 MHz	1.5 GHz (approx.)
On-chip RAM	2 KB SRAM	Up to 8 GB RAM (external)
On-chip Flash	32 KB Flash	Uses microSD card for storage
Operating System	No Operating System	Linux-based OS
Typical Use Case	Embedded and IoT applications	General-purpose computing
Power Consumption	Low	Higher than a microcontroller

Q13. Arduino Pin Types

Ans:

Pin type	description	Example	Iot use case
Digital Input	Reads HIGH or LOW signals from devices.	Push Button	Detecting button presses
Digital Output	Sends HIGH or LOW signals to control devices.	LED	Turning appliances ON/OFF
Analog Input	Reads varying voltage values (0–1023).	LDR Sensor	Measuring light intensity
PWM Output	Generates Pulse Width Modulation signals using `analogWrite()`.	Servo Motor or LED	LED brightness or motor speed control
I2C/SPI Pins	Used for communication with sensors and modules.	OLED Display, MPU6050	Connecting multiple IoT sensors

14) light traffic controller

Ans:

const int redLED = 2;

```
const int yellowLED = 3;
const int greenLED = 4;
const int button = 7;

void setup() {
  pinMode(redLED, OUTPUT);
  pinMode(yellowLED, OUTPUT);
  pinMode(greenLED, OUTPUT);
  pinMode(button, INPUT);
  Serial.begin(9600);
}

void loop() {

  if (digitalRead(button) == HIGH) {
    digitalWrite(redLED, HIGH);
    digitalWrite(yellowLED, LOW);
    digitalWrite(greenLED, LOW);

    Serial.print(millis());
    Serial.println(" ms : Pedestrian Crossing");

    delay(8000);
    return;
  }

  digitalWrite(redLED, HIGH);
  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, LOW);
  Serial.print(millis());
  Serial.println(" ms : RED");
  delay(5000);

  digitalWrite(redLED, LOW);
  digitalWrite(yellowLED, HIGH);
  Serial.print(millis());
  Serial.println(" ms : YELLOW");
```

```
  delay(2000);

  digitalWrite(yellowLED, LOW);
  digitalWrite(greenLED, HIGH);
  Serial.print(millis());
  Serial.println(" ms : GREEN");
  delay(4000);

  digitalWrite(greenLED, LOW);
}
```

19)What is the difference between analogWrite() and analogRead() in Arduino? What is PWM and why is it used? Give a practical IoT example for each.

Ans:

analogread()	analogwrite()
Reads an analog voltage from a sensor.	Generates a PWM signal to simulate analog output.
Returns values from **0–1023**.	Accepts values from **0–255**
Used with analog input pins	Used with PWM-enabled output pins.

What is PWM?

Pulse Width Modulation (PWM) is a technique used to control the average power supplied to a device by varying the width of the pulses.

Practical IoT Examples

- **analogRead():** Reading an LDR or soil moisture sensor.
- **analogWrite():** Controlling LED brightness or DC motor speed.

20) Explain the setup() and loop() functions in Arduino. What happens if you put a long delay() inside loop()? How does this affect sensor reading responsiveness? What is the non-blocking alternative?

Ans:

The `setup()` function runs only **once** when the Arduino is powered on or reset. It is used to initialize pins, variables, sensors, and communication interfaces such as the Serial Monitor.

The `loop()` function runs continuously after `setup()` finishes. It contains the main program logic that repeatedly reads sensors, processes data, and controls outputs.

If a long `delay()` is used inside `loop()`, the microcontroller stops executing other tasks during that time. As a result, sensor readings and responses to events become slow and unresponsive.

A better approach is to use the `millis()` function, which provides **non-blocking timing**. This allows the Arduino to perform multiple tasks simultaneously without pausing the entire program.

29) What is sensor calibration and why is it important in IoT systems? How would you calibrate an analog soil moisture sensor to give accurate percentage readings? Describe the two-point calibration method

Ans :

Sensor calibration is the process of adjusting a sensor's readings so that they match the actual physical values being measured. It improves the accuracy, reliability, and consistency of data collected by IoT devices.

For an analog soil moisture sensor, the **two-point calibration method** is commonly used:

1. **Dry Calibration:** Place the sensor in completely dry soil (or air) and record the sensor value. This represents **0% moisture**.
2. **Wet Calibration:** Place the sensor in fully wet soil or water and record the sensor value. This represents **100% moisture**.
3. Use these two reference values to convert all future sensor readings into a moisture percentage (0–100%) using a mapping formula.

This method ensures that the sensor provides more accurate soil moisture readings, helping IoT systems make better decisions for applications such as automatic irrigation.

30) Explain I²C protocol with respect to IoT sensor communication. What are SDA and SCL lines? How does I²C addressing work? Name 3 common IoT sensors that use I²C. (2 Marks)

Ans:

I²C (Inter-Integrated Circuit) is a serial communication protocol used to connect a microcontroller with multiple sensors and devices using only **two communication wires**.

- **SDA (Serial Data Line):** Carries the data between the master and slave devices.
- **SCL (Serial Clock Line):** Carries the clock signal generated by the master to synchronize communication.

In I²C communication, each device has a unique **7-bit or 10-bit address**. The master device (such as an Arduino or ESP32) sends the address of the desired slave device before exchanging data. This allows multiple sensors to share the same SDA and SCL lines without conflicts.

Examples of IoT sensors that use I²C:

1. **MPU6050** (Accelerometer and Gyroscope)
2. **BMP280** (Temperature and Pressure Sensor)
3. **BME280** (Temperature, Humidity, and Pressure Sensor)